

Encoding and Embedding

One Hot Encoding (word presence) → "Is the word present or not?"

Bag of Words (word count) → "How many times does the word appear?"

TF-IDF (count + importance weighting) → "How important is the word in this document?"

→ It reduces the weight of common words and increases the weight of more informative words

Embeddings (semantic + contextual understanding) → "What is the meaning of the text in context (word, sentence, paragraph, corpus)?"

One Hot Encoding

D1 → people watch movie

D2 → people watch cricket

D3 → people like movie

D4 → people like cricket

Vocabulary : {people, watch, like, movie, cricket}

	people	watch	like	movie	cricket
D1 →	1	1	0	1	0
D2 →	1	1	0	0	1
D3 →	1	0	1	1	0
D4 →	1	0	1	0	1

Document-level representation

D1 → people watch movie → [1, 1, 0, 1, 0]

D2 → people watch cricket → [1, 1, 0, 0, 1]

D3 → people like movie → [1, 0, 1, 1, 0]

D4 → people like cricket → [1, 0, 1, 0, 1]

Word-level (sequence) representation

D1 → people watch movie

[[1 0 0 0]

[0 1 0 0]

[0 0 1 0]]

D2 → people watch cricket

[[1 0 0 0]

[0 1 0 0]

[0 0 0 1]]

D3 → people like movie

[[1 0 0 0]

[0 0 1 0]

[0 0 0 1]]

D4 → people like cricket

[[1 0 0 0]

[0 0 1 0]

[0 0 0 1]]

Pros

Easy to implement
Simple binary representation
No training required
Direct mapping

Cons

Sparse Matrix Mostly zeros → memory waste
Sparse matrix means huge vector size but very little useful information."
Vocabulary size = 10,000 words
D1 → people watch movie
[1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ... up to 10,000]

High Dimensionality Vocabulary increase → vector size increase
No Semantic Understanding Similar words also seems unrelated
OOV Problem could not handle new words

D5 → people enjoy movie
people (known)
movie (known)
enjoy (NOT in vocabulary)

Bag of Words: Bag of Words captures how frequently a word is used in a meaningful context.
"If a word repeats more in a sentence, BoW assumes it is more important."

D1 → people watch movie and watch movie again

D2 → people watch cricket and watch cricket

D3 → people like movie and like movie a lot

D4 → people like cricket

Vocabulary

['again', 'and', 'cricket', 'like', 'lot', 'movie', 'people', 'watch']

	again	and	cricket	like	lot	movie	people	watch
D1 →	1	1	0	0	0	2	1	2
D2 →	0	1	2	0	0	0	1	2
D3 →	0	1	0	2	1	2	1	0
D4 →	0	0	1	1	0	0	1	0

Bag of Words is simple and powerful for counting words,
but it fails to understand meaning, order, and context.

Pros

1. Easy to implement

Very simple logic

Just count word occurrences

No complex math required

2. Captures word frequency

Shows how often a word appears

More occurrences → higher importance

3. Works well before embedding for classical NLP tasks

Useful for:

Text classification

Spam detection

Sentiment analysis (basic level)

4. No training required

No learning phase

Direct conversion from text → numbers

Cons

1. Ignores word order

Sentence meaning can change, but BoW won't notice

Example:

"dog bites man"

"man bites dog"

Same BoW representation

2. No semantic understanding

Similar words are treated completely different

Example:

"like" vs "love"

No relation captured

3. High dimensionality

Vocabulary size = vector size

Large dataset → huge vectors

4. Sparse representation

Mostly values are 0

Memory inefficient

Computation waste

5. OOV (Out of Vocabulary) problem

New words cannot be handled

If word not in vocabulary → ignored or error

6. Overemphasizes frequent words

Repeated words may dominate

Example:

"movie movie movie"

Gets high importance even if not meaningful

TF-IDF: Term Frequency (TF) Inverse Document Frequency (IDF)

D1 → people watch cricket

D2 → cricket watch cricket

D3 → people give comment

D4 → cricket give comment

Vocabulary: ['comment' 'cricket' 'give' 'people' 'watch']

TF(word, D) = (occurrences of the word in the sentence)
/ (Total words in the given sentence)

IDF(word) = $\log$ (Total number of sentence
/ Number of sentence containing the word)

	comment	cricket	give	people	watch
D1 →	0	$(1/3) \cdot \log(4/3)$	0	$(1/3) \cdot \log(4/2)$	$(1/3) \cdot \log(4/2)$
D2 →	0	$(2/3) \cdot \log(4/3)$	0	0	$(1/3) \cdot \log(4/2)$
D3 →	$(1/3) \cdot \log(4/2)$	0	$(1/3) \cdot \log(4/2)$	$(1/3) \cdot \log(4/2)$	0
D4 →	$(1/3) \cdot \log(4/2)$	$(1/3) \cdot \log(4/3)$	$(1/3) \cdot \log(4/2)$	0	0

D1 → people watch cricket
 D2 → cricket watch cricket
 D3 → people give comment
 D4 → cricket give comment

$\log(4/3) \approx 0.28768$
 $\log(4/2) = \log(2) \approx 0.69315$

	comment	cricket	give	people	watch
D1 →	0	0.096	0	0.231	0.231
D2 →	0	0.191	0	0	0.231
D3 →	0.231	0	0.231	0.231	0
D4 →	0.231	0.096	0.231	0	0

Why TF (Term Frequency)?

Because:

More occurrences = more importance in that document

Why IDF (Inverse Document Frequency)?

Because: Common words are not useful

Why multiply $TF \times IDF$?

Because:

TF → importance in document

IDF → importance in corpus

Why log in IDF?

Without log:

Rare words get extremely high values

Log:

Smooths values

Keeps balance

Log reduces extreme differences while keeping the importance order intact.

Log compresses large values so rare words don't overpower everything.

$$\text{IDF} = N / \text{DF}$$

Assume:

Total documents (N) = 1000

Case 1: Rare word

Appears in 1 document

$$\text{IDF} = 1000 / 1 = 1000$$

Case 2: Medium word

Appears in 10 documents

$$\text{IDF} = 1000 / 10 = 100$$

Case 3: Common word

Appears in 500 documents

$$\text{IDF} = 1000 / 500 = 2$$

Problem

Values are too extreme:

1000 vs 100 vs 2

Rare word dominates everything

Model becomes unstable

With Log

Formula:

$$\text{IDF} = \log(N / \text{DF})$$

Same cases:

Rare word:

$$\log(1000) \approx 6.9$$

Medium word:

$$\log(100) \approx 4.6$$

Common word:

$$\log(2) \approx 0.69$$

Now values become:

6.9 vs 4.6 vs 0.69

Pros

1. Captures importance of words
Not just count, but importance
Rare words → higher weight
Common words → lower weight
2. Reduces impact of common words
IDF reduces their weight
3. Better than BoW for relevance
Helps in:
Search engines
Information retrieval
Ranking documents
4. No training required
Same as BoW
Direct calculation
No model learning needed
5. Simple and interpretable
Easy to understand mathematically
 $TF \times IDF \rightarrow$ clear logic

Cons

1. Ignores word order
Same problem as BoW

Example:

"dog bites man"

"man bites dog"

Same representation

2. No semantic understanding
Cannot understand meaning

Example:

"car" vs "automobile"

Treated as completely different

3. Still sparse and high dimensional
Vocabulary size = vector size
Many zeros → inefficient

4. Doesn't handle context
Same word, different meaning → same vector

Example:

"bank" (river vs finance)

5. OOV problem
New words not in vocabulary → ignored

TF-IDF improves by adding importance but still fails to understand meaning and context.

Sentence 1:

"I like this movie"

Sentence 2:

"I love this film"

What happens:

OHE / BoW / TF-IDF

"like" $\neq$ "love"

"movie" $\neq$ "film"

→ Model thinks they are different

Embeddings

"like" $\approx$ "love"

"movie" $\approx$ "film"

→ Model understands similarity

